

Project Record

Every change, the work package it belongs to, the comparison against main, the architecture argument, and the known gaps

EdgePilot AI - Team 15

Documentation set, 26 August 2026

1 document in this volume

EdgePilot AI - Project Record

Every change made to this codebase, the part of the system it touched, what was wrong before it, and what the structure looks like now against the version the team shipped.

Project: EdgePilot AI - compare local and cloud AI deployment with real measurements. **Team:** 15. **This working copy:** Documents/Air On Accountant Backup/EdgePilot testing/app. **Compared against:** Desktop/Air on Accountants/edgepilot-ai-main (1)/edgepilot-ai-main, the team's repository after their updates. **Record written:** 23 August 2026, at commit b13af6e.

1. How to read this document

Section 3 is the fix list. Section 4 is the structural comparison. Section 5 is the argument for why this structure is the better one, and section 6 is the honest list of what it still does not do.

How a change is placed. Every fix below names the **work package** it lands in, not a person. The purpose is navigational: it tells you which part of the system to open, and therefore which conversation the change belongs to. The areas are the ones the repository already draws - the module a file sits in - so nothing here depends on anyone's recollection. Where a fix crosses two areas, both are named.

Verification method. Every "before" state in section 3 is a real file in the comparison copy on the Desktop, and every "after" state is a real file here. A claim of the form "this returned a constant" can be checked by opening both. Line counts come from diff between the two trees.

2. Where each work package lives

Use this to find the part of the system a change belongs to, and therefore which conversation it needs.

Benchmark and provider layer - measurement, the provider adapters, scoring and evidence.

src/modules/benchmark/, the benchmarks, providers, readiness, comparisons and local-runtime endpoints under src/app/api/v1/, scripts/benchmark/, scripts/evaluation/, docs/benchmark/, docs/local-model-setup.md.

Integration and API surface - routes, shared types, the database schema and the internal documentation.

src/app/api/v1/workloads/, src/shared/types/, prisma/, docs/internal/.

Product UI and benchmark dashboard - the four-step user journey and its styling. src/app/dashboard/, src/components/dashboard/, docs/dashboard/.

Vision benchmark - image classification end to end, including the dataset. src/modules/vision-benchmark/, src/app/vision-benchmark/, src/components/vision/, scripts/vision-benchmark/, docs/vision-benchmark/, datasets/.

Device knowledge and quality - removed in this pass; see change 22. The former src/modules/device/, src/app/api/v1/devices/, device-profile.json.

DevOps - the container, CI, the shared-database guards and the evidence page. Dockerfile, .dockerignore, .github/workflows/, scripts/db/, src/app/evidence/.

These are the divisions the repository itself already uses: AI_USAGE.md is organised one section per work package, and docs/benchmark/README.md and docs/dashboard/README.md are each written as a module guide.

3. The fix list

Forty-one changes, grouped by what they were about. Small ones are included - a one-character regex fix is in here, because it was the difference between a feature working and a feature that had never once worked.

3.1 Measurement correctness

These are the changes that decide whether a number in this application means anything.

1. hardwareFit was a constant. *Area: benchmark layer.* Every device on earth received `hardwareFit: 50`, from a Raspberry Pi to a threadripper. It is visible in the committed evidence: `evidence/benchmark/measured-ollama-llama3.2_latest.json` still records `"hardwareFit": 50`, because that artefact predates the fix. Replaced with `src/modules/benchmark/core/services/HardwareFitAssessor.ts` (229 lines, new), which scores *placement* from what the runtime reports, and `src/modules/benchmark/infrastructure/OllamaResidencyProbe.ts` (107 lines, new), which reads it. Ollama's `/api/ps` gives `size` (total resident bytes) and `size_vram` (how many of those are on the GPU); `size_vram / size` is the fit signal, and it is reported by the runtime rather than inferred by us.

2. Hardware fit deliberately does not score speed. *Area: benchmark layer.* Latency is already measured and already a readiness component. If hardware fit also punished CPU execution, a slow run would be penalised twice and the total would stop meaning anything. Recorded in the file's own header so the next person does not "fix" it back.

3. VRAM taken and spill share are now stated in words. *Area: benchmark layer.* The assessment says how much VRAM the model took and, on a partial offload, how many bytes did not fit and what percentage of the model that was. Where the runtime does not report card capacity, the assessment says so rather than presenting a share of an unknown total.

4. Cold start contaminated every first iteration. *Area: benchmark layer.* A model that is not resident has to be loaded from disk before it can answer. Measured on the same 8B model in the same session: time-to-first-token was **36,445 ms cold** and **369 ms warm** - a factor of 99 - and readiness swung from 75 to 90 on load state alone. Averaging that first iteration into a five-iteration mean does not describe the model; it describes whether someone had used it recently. Fixed in `BenchmarkRunner.ts`: the runner now runs $N+1$ iterations and discards the first. The user is told their chosen iteration count will be raised by one, the discarded cold boot is reported with its own duration and its own row in the results, and the database records it as `warmup: true` rather than deleting it.

5. The success chain counted the discarded iteration. *Area: benchmark layer.* After change 4, a request for three iterations reported "4/4 iterations succeeded". The measured set is now `responses.slice(1)` and success, failure-code dominance and the summary all read from that. The fix went into the runner rather than the test that caught it.

6. The residency probe result was used without a null guard. *Area: benchmark layer.* The catch produced the correct value for the wrong reason, which would have masked any other fault in the probe. Now before `=== null ? null : before.residentBytes !== null && before.residentBytes > 0`.

7. Privacy was averaged into the readiness score. *Area: benchmark layer.* A coarse label was mapped to 30/60/100 and blended with latency and reliability. That is a category error twice over: it implies a vendor is "twice as private" as another, and it lets a fast provider average away a data policy that should have disqualified it outright. `ReadinessCalculator.ts` dropped from a five-term average to four measured performance terms, and privacy became an ordinal class produced by `PrivacyAssessor.ts` (314 lines, new) that annotates or disqualifies a recommendation instead of nudging a number.

8. A mean was being reported as a result. *Area: benchmark layer.* `ComparisonReport.ts` (506 lines, new) checks every dimension against the per-iteration samples behind it. If the entrants' observed ranges overlap, no winner is declared. At five iterations a 10% gap in means routinely sits inside a 3x spread. A worked consequence: after cold starts were removed, latency did **not** become a cleanly separated dimension - separability *fell* from 0.82 to 0.58, because removing the cold start shrank the gap more than it shrank the spread. The report says so rather than declaring a winner.

9. Two local models were about to be compared in parallel. *Area: benchmark layer.* For two cloud providers, parallel is correct. For two local models it destroys the measurement: both want the same GPU, the runtime may evict one to make room for the other, and the latency that comes back describes contention.

ComparisonPlanner.ts (141 lines, new) decides execution mode before anything runs, and RunComparison.ts honours the plan instead of always calling Promise.all.

10. Nothing stopped a text model being compared to an embedding model. Area: *benchmark layer*.

ModelModality.ts (167 lines, new) classifies from Ollama's families array. Vision (clip, mllama, llava, qwen2vl, vision) and embedding (bert, nomic-bert, gte, jina-bert) are identified positively; text is the residual and is always marked inferred, because "no vision family present" is not proof of anything.

11. task_type was a column nothing read. Area: *benchmark layer, over the integration layer's workload schema*. You could register an image_recognition workload and benchmark an embedding model against it and no part of the system objected. TaskCompatibility.ts (174 lines, new) turns the declaration into a constraint. The rule is deliberately asymmetric: a vision model can do text, a text model cannot do vision, an embedding model can do neither.

12. Timeouts were too low for a cold local model, and for cloud. Area: *benchmark layer*. Raised for the local path so a genuine cold load is not recorded as a timeout, and raised for the online path in the same pass.

13. code_generation no longer implies the code works. Area: *benchmark layer*.

shapeOf('code_generation') returns text that says so: nothing in this system compiles or runs the output, so "it produced code" is not "the code works".

3.2 Identity, ownership and privacy

14. Users were being asked to paste UUIDs by hand. Area: *dashboard, and integration*. The setup step had a field for a workload UUID because the endpoint that should have created one did not persist. Replaced by src/components/dashboard/session.ts (92 lines, new): the browser generates a durable id, stores it in localStorage, sends it as x-edgepilot-session on every request, and **never renders it**. A user should no more be asked to see their own row-ownership key than to type their own cookie.

15. POST /api/v1/workloads never called the database. Area: *integration*. The committed version returned the literal string temp-workload-id, so nothing the dashboard saved reached Postgres and every benchmark run afterwards failed with 404 Workload not found. The route is now 126 lines against the original 53 and writes real rows. The original is kept beside it as route.ts.scaffold.bak and is referenced by name in the new file's header.

16. Every row was attributed to one shared user. Area: *integration*. Two routes attributed everything to local-sandbox@edgepilot.invalid. Harmless on one laptop; a data leak the moment it is hosted, because GET would hand every visitor every other visitor's workloads. src/lib/sessionOwner.ts (76 lines, new) upserts a User per session id. It fails closed: a request without the header gets 400, never a fallback to a shared user.

17. Session activity had nowhere to be recorded, and no redaction rule. Area: *benchmark layer*.

src/core/logging/SessionLog.ts (407 lines, new) records what the application did, exportable as JSON, with no account attached. Prompt content is never recorded by default - only a length and a digest. A log that wrote out the same prompt the egress warning had just flagged would have defeated its own warning. Logging is opt-in by header: a request with no session id is not logged anonymously, it is not logged at all.

18. The privacy report arrived after the prompt had already left. Area: *benchmark layer*. egress-warning.ts (133 lines, new) answers the question before the run: is this prompt about to leave the machine, and to whom. It fails closed on a null endpoint.

19. The egress warning object contained the prompt text. Area: *benchmark layer*. Found by a test written in the same session. An object whose job is to warn that text is about to leave the machine must not carry a copy of that text. It now reports the character count and nothing else. An older assertion in privacy-assessor.test.ts had pinned the opposite behaviour and was reversed, with the reasoning recorded at the call site.

20. The privacy catalogue ships empty on purpose. Area: *benchmark layer*. Every value in privacy-catalogue.ts starts null and unverified. These are policy facts from terms of service, they differ between free and paid tiers, and they change without notice. Filling them in from memory is how an evidence-based tool starts publishing stale claims confidently. Filling them in is a human task and is listed in section 6.

21. The login page was removed. *Area: integration.* There are no accounts, so an auth surface was a door onto an empty room. `src/app/auth/` and `src/app/api/auth/` are gone.

3.3 Removing the device module

22. The device profile was a client-side self-report that nothing could check. *Area: device.* The user typed how much RAM they had; the application stored it; no measurement ever used it. Once hardware fit became `size_vram / size` read from the runtime, the typed value had no consumer and could only disagree with reality. Removed to the roots rather than left nullable: the UI panel, the API route, the module, the types, the tests, and the database.

23. The database migration. *Area: device; the migration came out of the benchmark and DevOps work.* `prisma/migrations/20260823035500_drop_devices/` drops two foreign keys, the `benchmarks_device_id_idx` index, the `benchmarks.device_id` column and the `devices` table. Tested on the local sandbox database before it was applied to the shared Neon instance, and applied there with `migrate deploy - never migrate dev`, which can offer to reset a shared database.

24. Two dev scripts still referenced devices, and next build caught them. *Area: benchmark layer.* `scripts/dev/show-last-run.ts` read `run.deviceId` and `scripts/dev/seed-demo-rows.ts` called `prisma.device.create`. Jest never touched either file. This is the reason `npm run build` is in the verification list and not just `npm test`.

3.4 The vision benchmark

25. The live evidence path had never worked, once, ever. *Area: vision.* `evidence-store.ts` validated filenames against `/^[a-z0-9._-]+\.[a-z0-9]+$/` - lowercase only - and then built the filename from an ISO timestamp containing an uppercase T and Z. Every live run failed to write its evidence. One `.toLowerCase()` fixed it. It is the smallest change in this document and it is the reason any live measurement exists at all.

26. The first vision sample paid the cold-start cost. *Area: vision; the pattern came from the benchmark runner.* The executor now runs a warm-up classification before the measured set. Median latency across the dataset fell from 1322.1 ms to 529.6 ms between the two live runs - the same model, the same images, the difference being that sample one was no longer carrying the model load.

27. The warm-up was wrong three times before it was right. Recorded because the mistakes are instructive. It consumed a deterministic test clock (fixed by calling `provider.classify()` directly rather than the timed wrapper); it fired on cloud providers, which meant a billable call and one extra image sent to a third party (fixed: local providers only, plus an explicit `warmUp?: boolean`); and it ate scripted responses in two existing tests (fixed by opting those two out with a comment, and adding two new tests that cover the behaviour properly).

28. Users can now bring their own dataset, and it never leaves their machine. *Area: vision.* `src/components/vision/DatasetUpload.tsx` (403 lines, new) takes a folder-per-class upload. `browser-image-processor.ts` (181 lines, new) resizes and hashes in the tab; `browser-ollama-provider.ts` (171 lines, new) talks from the tab straight to the user's own Ollama on localhost. No uploaded image ever reaches the EdgePilot server. That is not a convenience decision - benchmarking confidential data is this project's stated non-goal, and shipping images to a server to be resized would have contradicted it.

29. The browser processor's version string is deliberately different. *Area: vision.* It is `browser-canvas-png-v1`, not `sharp-png-v1`. Canvas and sharp do not resample or encode PNG identically, so two runs prepared differently must not be comparable by accident. Evidence records the preprocessing version for exactly this reason.

30. Arbitrary labels and licences. *Area: vision.* The metrics schema pinned `perClass` to `.length(VISION_LABELS.length)` - seven. Any dataset with a different number of classes ran to completion and *then* failed validation. Six fields were widened in one sweep, TypeScript interfaces and zod schemas together, after three separate build round-trips caused by widening one without the other. All three pre-existing evidence files were re-parsed afterwards to confirm backward compatibility.

31. The prompt has to come from the dataset. *Area: vision.* A deliberate failure run proved it: with the built-in prompt and a custom two-class dataset the result was 0% accuracy and 100% invalid output.

`buildVisionPrompt(labels)` builds the closed label set from the dataset. Re-run with `red_panel/green_panel: 100%`, passed.

32. A loose file at the chosen root took the root folder's name as its class. *Area: vision.* `labelOf` now requires three path parts, so only `root/class/file` is accepted.

33. Four declarations block the run. *Area: vision.* SPDX licence text plus three privacy checkboxes. A dataset with no stated licence and no confirmation of what is in the images is not a dataset anyone should publish a metric from.

34. The real llava measurement, recorded as a failure. *Area: vision.* 21 samples, 14 correct: **66.7% accuracy, macro-F1 0.667**, 0% invalid output, 100% request success, median 529.6 ms, p95 721.4 ms. It **fails** its own gates (0.80 accuracy, 0.75 macro-F1) and the artefact records `"passed": false`. The per-class matrix closes exactly: `hardhat` has three false negatives against `safety_cone`'s three false positives - all three `hardhats` were called `safety_cones`, and `hardhat` scores F1 0.0. Two gloves and two masks were called `hardhats`. This is kept in the repository deliberately; see change 40.

3.5 Build, evidence and repository hygiene

35. .next held 142 stale files pointing at an old Downloads path against 75 correct ones, which is what the React Client Manifest errors were. Cleared.

36. .dockerignore was excluding something the build needed. *Area: DevOps.* Fixed and proven: `docker build -t edgepilot-ai:test .` completes in 154.8 s with `RUN npm run build` succeeding inside the image.

37. The evidence page traced the whole project into the standalone bundle. *Area: DevOps.* A dynamic `path.join(process.cwd(), root)` inside a loop defeats Next's file tracing. Rewritten with literal path segments.

38. /evidence exists as a page. *Area: DevOps.* `src/app/evidence/page.tsx` (286 lines, new) leads every artefact with `what_this_proves` and `what_this_does_not_prove` before showing a single number.

39. The ten-case evaluation matrix was missing. *Area: benchmark layer.* `scripts/evaluation/ten-case-matrix.ts` (340 lines, new), run with `npm run eval:matrix`, drives real module code - the same zod schemas, normaliser and readiness calculator the application uses - across 10 cases in 6 categories (1 normal, 3 malformed, 1 ambiguous, 3 injection, 1 missing-evidence, 1 provider-failure) and exits non-zero on failure. Current artefact: all 10 behaved as documented, no failures. Case 9 exists to assert that a missing hardware fit stays `null` and is never rendered as `0`.

40. Working backups and the one real measurement were both mishandled by git. *Area: DevOps.* Nineteen `.bak` files sat beside real source, and ten of them were already tracked - so a `.gitignore` rule alone would not have removed them. Eighteen were cleared out and `*.bak` was added to `.gitignore`. In the same rule set, `/evidence/vision-benchmark/live-*.json` was silently ignoring the llava run - the first real measurement the project produced. Both decisions are now encoded as negation rules with the reasoning written beside them, rather than as `git add -f` flags someone has to remember.

41. scripts/db/ guards the shared database. *Area: DevOps.* `dotenv -e .env.neon -- prisma migrate deploy` fails *silently* when the file is absent: `dotenv` loads nothing, `Prisma` falls back to `.env`, and a command named "neon" runs against localhost while reporting success. `require-env-file.mjs` refuses to proceed unless the file exists, parses, and names a host that is not local; `neon.mjs` reads the file itself and prints the destination host before doing anything.

4. This structure against the team's

Compared with `Desktop/Air on Accountants/edgepilot-ai-main (1)/edgepilot-ai-main`, excluding `node_modules`, `.next`, `.git`, `package-lock.json` and `.env`. Fifty-five files differ, fifty-two paths exist only here - two of them local scratch (`.env.neon`, `_to_delete/`) rather than project content - and twenty-six exist only there.

4.1 What is gone

Removed	Why
src/modules/device/, src/app/api/v1/devices/, src/shared/types/device.ts, tests/device/	Self-reported hardware that no measurement consumed - see change 22
src/app/auth/, src/app/api/auth/	There are no accounts; ownership is a browser-generated session id
sentry.client.config.ts, sentry.edge.config.ts, sentry.server.config.ts	Three config files for a service with no DSN and no project behind it
src/core/rate-limit/, src/core/security/	A rate_limits table and a middleware layer guarding endpoints that have no authentication and no public URL
src/data/	A parallel data-access path beside src/modules/*/infrastructure/repositories/
src/components/ErrorBoundary.tsx, Loading.tsx, LoadingStates.tsx, providers/	Three overlapping loading components; the App Router's own loading.tsx and error.tsx conventions cover it, and StateViews.tsx holds the rest
src/core/logging/logger.ts	Superseded by SessionLog.ts, which has a redaction rule and an export format
AUDIT_REPORT.md, FINAL_AUDIT_REPORT.md, PRODUCTION_AUDIT_REPORT.md, REMAINING_PRODUCTION_GAPS.md, TEST_DOCUMENTATION.md	Five root-level audit documents describing the same codebase at five moments, none marked superseded. Replaced by this one record plus per-folder documentation
public/index.html, public/favicon.svg	A static page shadowing an App Router route
docs/deployment.md	Described a deployment that does not exist yet; the honest version is section 6

4.2 What is new

Added	What it does
src/modules/benchmark/core/services/ - 8 new files, 1,761 lines	Hardware fit, modality, task compatibility, privacy class, egress warning, policy catalogue, comparison planner, comparison report
src/modules/benchmark/infrastructure/OllamaCatalog.ts, OllamaResidencyProbe.ts	Runtime status, installed models, and the memory reading that hardware fit is computed from
src/app/api/v1/comparisons/, local-runtime/, session-log/, session-log/share/	Four endpoints the original did not have
src/app/evidence/page.tsx	Artefacts readable without running anything
src/core/logging/SessionLog.ts, sessionLogStore.ts	Redacting, opt-in, exportable session record
src/lib/sessionOwner.ts, src/components/dashboard/session.ts	Account-free row ownership
src/components/dashboard/InstalledModels.tsx, VisionHandoff.tsx	Always-visible model panel with per-step filtering; handoff into the vision dashboard
src/components/vision/DatasetUpload.tsx and the two browser vision adapters	Bring-your-own dataset that never leaves the machine
scripts/db/, scripts/dev/, scripts/evaluation/	Shared-database guards, local helpers, the ten-case matrix
tests/core/, tests/evaluation/, plus new benchmark and dashboard suites	322 tests across 29 suites, from 259
4 Prisma migrations	Nullable hardware fit and shared findings; optional email and session owner; drop devices; warm-up iteration

Added	What it does
AI_USAGE.md	Required submission item, one section per work package

4.3 What changed in place

The twenty largest, by lines differing:

File	Theirs	Ours	Lines differing	Work package
src/app/globals.css	418	27	431	Dashboard
src/app/dashboard/page.tsx	692	15	697	Dashboard
src/components/dashboard/SetupPanel.tsx	322	201	379	Dashboard
src/app/vision-benchmark/page.tsx	73	244	293	Vision
src/modules/benchmark/application/services/BenchmarkRunner.ts	469	662	269	Benchmark
prisma/seed.ts	127	56	137	Integration
src/app/api/v1/workloads/route.ts	53	125	126	Integration
src/app/page.tsx	97	36	123	Dashboard
src/components/dashboard/RunPanel.tsx	214	315	119	Dashboard
src/components/dashboard/ProviderPanel.tsx	219	294	113	Dashboard
src/modules/benchmark/core/services/ReadinessCalculator.ts	40	88	110	Benchmark
src/components/dashboard/DashboardApp.tsx	134	203	91	Dashboard
src/modules/benchmark/application/use-cases/RunBenchmark.ts	316	308	90	Benchmark
tests/vision-benchmark/vision-execution.test.ts	301	361	86	Vision
prisma/schema.prisma	167	199	82	Integration
src/components/dashboard/api.ts	202	242	78	Dashboard
tests/benchmark/run-benchmark.test.ts	488	483	73	Benchmark
src/components/dashboard/RunResults.tsx	358	413	69	Dashboard
src/modules/vision-benchmark/application/executor.ts	159	222	65	Vision
src/modules/vision-benchmark/core/types.ts	159	201	62	Vision

src/app/dashboard/page.tsx going from 692 lines to 15 is not deletion. The route file is now a route file - metadata and a mount point - and the 692 lines of behaviour live in src/components/dashboard/, where they can be tested without a request.

5. Why this structure is the better one

5.1 The boundaries are real, not decorative

Both versions describe themselves as hexagonal. In this one the description holds when you check it. src/modules/benchmark/core/ imports nothing from infrastructure/; the ports are two interface files, AIPROvider.ts and BenchmarkRepository.ts; and infrastructure/container.ts is the single place where an adapter is constructed. No API route in this tree builds a provider, reads an environment variable, or touches Prisma. That is what makes the provider layer testable without a network: tests/benchmark/helpers.ts injects a fake fetch and a stepped clock, and the suite is deterministic because nothing underneath it can reach out.

The version with src/data/ beside src/modules/*/infrastructure/repositories/ had two answers to "where does data access live", which is the same as having none.

5.2 Every number carries its provenance

measured, derived, declared, unverified are values in the type system, not comments. A reader can tell a stopwatch reading from an arithmetic derivation from a placeholder without leaving the JSON. The rule that follows from it is the one that costs the most to keep: **null is never coerced to 0**. Reporting 0 ms for "we never got an answer" is the single most misleading number this system could produce, and there is a test file (tests/dashboard/format.test.ts) whose entire job is to stop it happening in the UI, and a case in the ten-case matrix that asserts it in the data.

A structure where hardwareFit was the constant 50 could not have supported that rule, because the field looked like a measurement and was not one.

5.3 It deletes

The clearest structural difference is that one tree accumulates and the other removes. Five audit reports at the root, three Sentry configs with no DSN, a rate limiter in front of unauthenticated endpoints, three loading components - each of those was reasonable when it was written, and none of them was ever taken back out. Dead structure is not free: it is read by reviewers as current, it appears in searches, and it makes the live parts harder to find.

Removing the device module cost a migration, a UI change, a module deletion and a test deletion. Leaving the column nullable would have cost nothing today and misled every reader afterwards.

5.4 The tests aim at refusals

The interesting question for a measurement tool is not "does it work" but "what does it refuse to say". So the suite is written against refusals: refuse to call a provider for a workload that does not exist; refuse to attribute a run to the wrong user; refuse to discard a measurement because the database went down mid-run; refuse to declare a comparison winner inside the noise; refuse to report a mean when the only iteration was a cold start.

That framing is also why `npm run build` is a required check alongside `npm test`. Jest type-checks only what it imports. `next build` type-checks the whole project, and it is what caught the two dev scripts in change 24 and the six schema-versus-interface mismatches in change 30 - nine defects that a green test suite could not see.

5.5 Evidence is an artefact, not a claim

evidence/ holds files a reviewer can read without installing anything, and each leads with what it does **not** prove. The vision run that fails its own gate is committed *because* it fails: a benchmarking tool whose only published measurement is a pass is not demonstrating a benchmark, it is demonstrating selection.

5.6 The client/server line is load-bearing

infrastructure/config.ts refuses to evaluate in a browser bundle, because every value it holds is a secret or a host. The two browser vision adapters exist for the inverse reason: images must *not* travel to a server, so the processor and the provider run in the tab. Both directions come from the same rule - decide where a thing is allowed to be, then make the wrong location fail loudly.

6. What this still does not do

Listed so nobody has to discover it in a review.

- **No cloud provider path has ever executed.** One Gemini or Groq key would light up the egress warning, tier-aware privacy and free-versus-paid comparison in a single run.
- **The privacy catalogue is still null and unverified.** Deliberate - see change 20 - and a human task.
- **official_source never reaches a recommendation**, which leaves an acceptance criterion open.
- **quality_score exists nowhere**, and is a required structured field.
- **No public URL.** The Docker image builds; nothing is hosted.
- **~~No UI for comparisons, session-log or session-log/share~~ Closed.** /compare drives the comparison engine, /history drives the session log and its consent-gated share, and /evaluation renders the ten-case matrix. All three endpoints had working backends and no way in.

- **No adversarial or injection tests in the Jest suite.** The ten-case matrix covers the artefact side; the unit suite does not.
 - **Two deliberate conflicts with the specification**, recorded rather than hidden: the device profile registry was a mandatory MVP item and was removed, and `readiness_score()` was specified to weigh privacy, which it no longer does. Sections 3.1 and 3.3 are the defence of both.
 - **docs/internal/database.md is stale** - line 147 still says users own devices, and it omits `shared_findings`, `session_id`, `warmup` and `privacy_class`.
 - **device-profile.json is still at the root**, referenced only by `.dockerignore` and two historical evidence files.
 - **describeEgressWarning is tested in two files.** Flagged and left alone deliberately.
-

6a. What changed after this record was written

The record above describes the state at commit `b13af6e`. Three further passes landed on top of it, and the parts that change what section 6 claims are here rather than rewritten into it - a record that quietly edits itself is not one.

The interface arrived. A UI/UX pass added `/compare`, `/evaluation`, theming and an arcade navigation, and closed the "endpoint works, nothing calls it" gap for three endpoints at once. Its only reach into measurement code was `OllamaCatalog`, which gained a per-model resident flag read from `/api/ps` - null when the call fails, never conflated with "not loaded".

The vision workload became runnable from the page. `/vision-benchmark` had a server path and a browser path for your own images, and no button for the dataset the project ships with. It is a server action, off by default behind `VISION_BENCHMARK_IN_APP`, and it writes nothing server-side - the evidence is returned for the browser to keep, so `evidence/vision-benchmark/` still means one thing: the reference measurements committed with the project.

Vision runs gained the metrics the text benchmark had. Hardware fit, read by the same residency probe and scored by the same assessor; and the timings Ollama already returned on every reply and the provider was discarding - prompt-eval time, token counts, tokens per second, model load. Deliberately **no TTFT**: that requires streaming, the vision provider sends `stream: false`, and there is no first-token moment to observe. `promptEvalMs` is the closest honest quantity and does not borrow the name.

Every run is kept. `/history` shows text and code runs, comparisons and vision runs together, in the browser rather than on the server. Before it, the dashboard dropped a completed run on navigation and a comparison - the most expensive operation here - survived nowhere at all.

One correctness bug found and fixed. The comparison accepted the same model twice. Entrants are labelled `provider + model` and the report's tally is keyed by that label, so duplicates collided onto one key: it would have declared a winner between a model and itself, decided by whichever run happened to be faster. Refused in `planComparison`, mirrored client-side, pinned by two tests - one that it refuses duplicates, one that `llama3.2:1b` against `llama3.2:3b` still runs, because the rule is about identity and not similarity.

Licensing changed. This work is AGPL-3.0-or-later; the imported baseline keeps its MIT notice, which is a licence obligation rather than a statement about who wrote what.

Still open from section 6: no cloud provider path has executed, the privacy catalogue is still unverified, `official_source` and `quality_score` remain, there is no public URL, and `docs/internal/database.md` is still stale.

6b. Hosted mode

A fourth pass, made for one reason: the site is going to live on Vercel with a Neon database, and most of the Ollama plumbing had been written for a machine where the page and the runtime were the same computer. Section 6a's "no public URL" was about to close, and closing it would have broken every local run for every visitor.

The problem, stated plainly. Every Ollama call except the upload-your-own- dataset path ran on the server and read `OLLAMA_HOST`, which defaults to `localhost`. On a laptop that is the same Ollama the browser sees. Hosted, "localhost" is a container in a datacentre with nothing listening, and a server cannot reach a visitor's laptop. The dashboard, the comparison and the built-in vision run would all have reported "not reachable" for everyone.

Ollama moved into the browser. `infrastructure/browser-ollama.ts` reuses `OllamaCatalog`, `OllamaProvider` and `OllamaResidencyProbe` unchanged - they were plain fetch with no server dependency - and runs them from the tab against the visitor's own machine. The tab does only what it alone can do (the HTTP calls) and sends the raw measurements to the server in a recorded field on the same benchmark and comparison requests. There a `RecordedProvider` replays them through the unchanged `BenchmarkRunner`: same cold-start isolation, same summary, same hardware fit from the residency readings the browser took, same readiness score, same database row, same activity log. A run measured in the browser and one measured on the server are the same kind of evidence, which is what keeps them comparable. No fallback for a recorded run: a cloud model is not a substitute measurement of the visitor's machine. The built-in vision run followed the same path - two new routes serve the dataset description and its 21 images, and the shared executor runs in the browser against the visitor's Ollama.

What cannot be made invisible. Ollama refuses any website it was not told to trust, which is the right default and not ours to override. A visitor allows the site's origin once (`OLLAMA_ORIGINS`), per operating system. `/setup` gives that the room it needs - OS tabs, the exact command with the real origin filled in, a copy button, a connection check - and every place a visitor can hit the refusal (the installed-models panel, the vision page, the home page) points there in one line.

Cloud providers gained real model lists. The provider step used to claim a cloud vendor "does not publish its catalogue" and offered free text with a hardcoded suggestion list that had already drifted from `.env.example`. Both vendors publish it under the same key; `CloudCatalog` asks, narrows to text generators, and marks which accept an image. The dashboard, the comparison and the vision page all pick from real names now, and the placeholder option can no longer be submitted.

Visitors can bring their own keys. Stored in their browser, sent as two request headers, laid over the server's configuration for that one request through a registry that is never cached, never logged, never stored, never echoed. A visitor without keys still uses the site's, and those runs are capped per visitor per hour (core/quota) so the shared quota cannot be drained by one person clicking all evening.

Vision on Gemini and Groq. A Groq vision adapter (OpenAI chat dialect, image as a data URL) joined the Gemini one; the server action takes the provider, the model and the visitor's keys as arguments, because a server action carries no request headers; cloud runs classify seven images at a time so 21 fit a serverless time limit, while a local run stays strictly sequential - two requests to one GPU measure contention, not the model. Committed reference rows stay as they were: controlled Gemini, controlled Ollama, live llava.

Removed. `/api/v1/health/database` and the panel on `/history` that called it. Hosted, an unauthenticated endpoint announcing row counts and migration names to whoever asks is not a diagnostic, and the panel was a debugging aid on a user-facing page. Both are kept outside the repository for local use.

Smaller things that would have broken the deployment. The evidence commit hash falls back to `VERCEL_GIT_COMMIT_SHA` (no `.git` on Vercel); `next.config.mjs` traces `datasets/` into the build (a `readFile` path is not an import, and without this every hosted built-in run fails with `ENOENT`); the vision page sets `maxDuration`; `.env.example` ends with the list a deployment needs, including `BENCHMARK_FALLBACK_ORDER=groq,gemini` so a failed cloud run does not fall back to an Ollama that is not there.

Section 6 after this pass: the cloud provider path now executes from the page (model lists confirmed against live keys; a Groq vision run awaits a key with access to an image-capable model); the privacy catalogue is still unverified; `official_source` and `quality_score` remain; `docs/internal/database.md` is still stale. The public URL is the next step, and the code is now written for it.

7. Verification state at the time of writing

Check	Command	Result
Lint	<code>npm run lint</code>	0 problems
Tests	<code>npm test</code>	348 passed, 33 suites (after 6b)
Build	<code>npm run build</code>	clean, 14 routes
Container	<code>docker build -t edgepilot-ai:test .</code>	succeeded, 154.8 s
Evaluation matrix	<code>npm run eval:matrix</code>	10/10 cases behaved as documented
Vision, live	<code>vision:run:ollama --model=llava:latest</code>	ran, 66.7%, failed its gate , recorded

Two credentials for the shared Neon database were exposed during the work that produced this record and should be rotated. `.env` and `.env.neon` are ignored by `.gitignore` line 28 (`.env*`); `git status --short` must show nothing matching `.env` before any commit.

8. Where the rest of the documentation is

Every folder in this project carries a `FOLDER.md` describing what it is, what happened in it, what each file inside it does, and which other folders it is connected to. Those cross-references are written in both directions, so a trail can be followed either way.

Start at `src/modules/benchmark/FOLDER.md` for the measurement core, or `src/app/FOLDER.md` for the request path.

Eighty-eight folder documents, with cross-folder links, every one of them resolving in both directions - if folder A points at folder B, B carries a "Referenced from" entry pointing back at A.

A PDF set lives in `Documents/AoC Documents`: this record on its own, twelve volumes grouped by area, and one complete 112-page bundle containing everything.